

TradeQuote — AI Back Office for Small Contractors

Build Plan & Architecture (Claude Code reference doc)

One-liner: Turn a tradesperson's site-visit voice note into a professional, branded quote — then handle invoicing and payment chasing — powered by a per-contractor knowledge layer that learns their rates, suppliers, and past jobs.

Target user: 2–10 person trade businesses (builders, electricians, plumbers, roofers) currently running on WhatsApp, paper, and gut feel.

Product Phases

Phase 0 — Voice-first setup layer (onboarding)

A conversational setup wizard — voice wherever possible, tap-to-answer fallback. Claude asks, contractor talks, app confirms each answer on screen before saving. Target: fully set up in under 10 minutes from the van.

Setup interview covers:

- Company name and company number (auto-verify against Companies House API — free, confirms name/number/status)
- VAT registered? (yes/no → VAT number if yes)
- Day rate, plus variables: rate per trade/role, overtime or weekend rate, minimum call-out charge, travel charge
- Who's going onsite — team members, their roles, and their individual day rates
- Materials pricing approach:
 - Default: pull list prices from merchant sites (Travis Perkins, Screwfix, Toolstation, Jewson) via scheduled scrape or product-search API where available
 - Contractor confirms which merchants they actually use
- Trade discount? — per-merchant discount % applied on top of list prices (e.g. "15% at Travis Perkins")
- Branding: logo upload (or generate one in Phase 3), brand colour, quote footer terms

Flow: each question is one voice exchange → Claude extracts the answer into a structured field → shown as a card the contractor taps to confirm or re-record. Skippable questions get sensible defaults flagged as "assumed — update anytime in Settings."

Materials price scraping notes:

- Start with 2–3 merchants max; a nightly scrape of the ~200 most-quoted SKUs per trade beats trying to index whole catalogues
- Cache prices with a "price as of [date]" stamp on quotes to protect the contractor
- Check each merchant's terms; where scraping is disallowed, fall back to contractor-confirmed prices that the knowledge layer remembers
- Trade discounts stored per contractor per merchant and applied automatically to material line items

Phase 1 — Quoting MVP (build this first)

- Contractor records a voice note during/after a site visit
- App transcribes it, extracts job details, and drafts a structured quote (labour, materials, travel, day rates)
- Contractor reviews/edits on their phone, then sends a branded PDF quote to the customer
- Success metric: quote created in under 5 minutes vs. an evening at the kitchen table

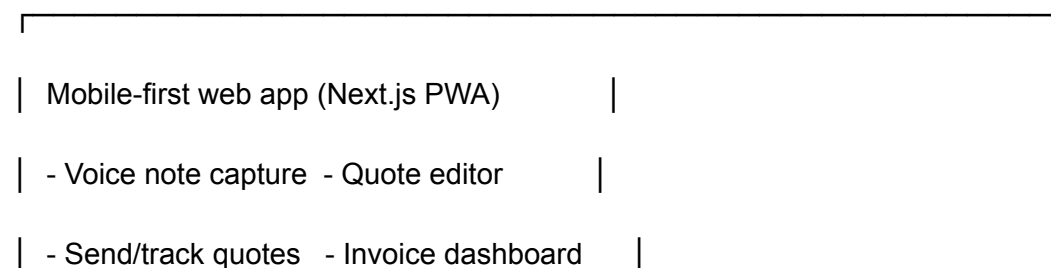
Phase 2 — Invoicing & payment chasing

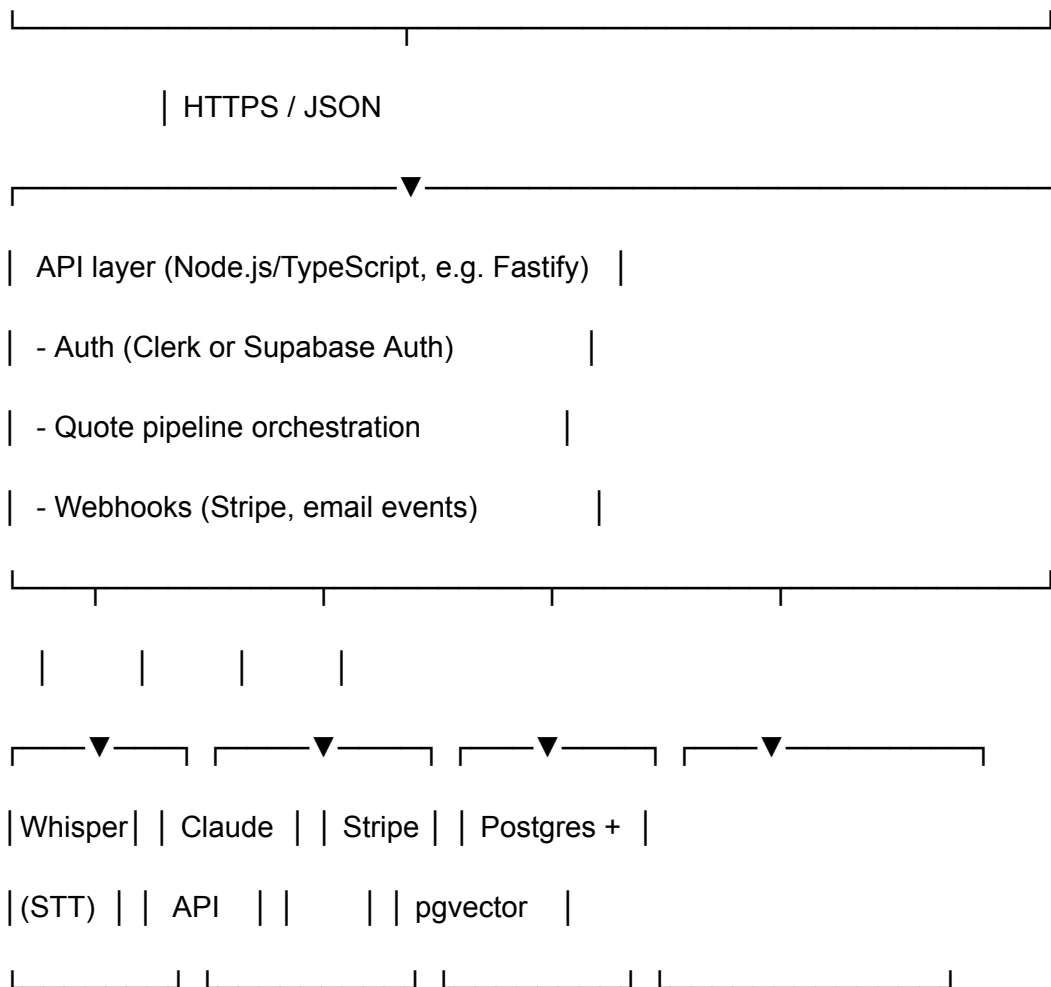
- Convert accepted quotes into invoices (deposit / staged / final)
- Stripe payment links embedded in invoices
- Automated, polite chase sequences for overdue payments (email/SMS)

Phase 3 — Knowledge layer

- Per-contractor semantic memory: historical quotes, preferred suppliers, material prices, typical day rates, travel charges
- Each new quote gets smarter — "you quoted £X/m² for similar plastering in March"
- Later: procurement suggestions, logo/branding generation, simple cashflow view

Architecture Overview





Stack choices (light, boring, fast to ship)

Layer	Choice	Why
Frontend	Next.js PWA, Tailwind	One codebase, installable on phones, no app-store friction
Backend	Node.js + TypeScript (Fastify or Next API routes)	Same stack as Carly — familiar, fast to prototype
Database	Postgres (Supabase) + pgvector	Relational data + vector search in one place, no separate vector DB to run
Auth	Supabase Auth or Clerk	Solved problem, don't build it

Layer	Choice	Why
Speech-to-text	OpenAI Whisper API (or on-device fallback)	Handles noisy site audio, trade jargon reasonably well
LLM	Claude API — claude-sonnet-4-6 for the quote pipeline; claude-haiku-4-5 for cheap classification/chasing copy	Sonnet for quality reasoning on quotes; Haiku where cost matters. Docs: https://docs.claude.com/en/api/overview
Payments	Stripe (Invoicing + Payment Links)	Invoices, links, and webhook events out of the box
PDF generation	react-pdf or Puppeteer HTML→PDF	Branded quote/invoice PDFs
Email/SMS	Resend (email) + Twilio (SMS chasing)	Simple APIs, cheap at low volume
Hosting	Vercel (app) + Supabase (data)	Zero-ops for MVP

Core Pipeline: Voice Note → Quote

1. **Capture** — browser MediaRecorder uploads audio to storage (Supabase Storage)
2. **Transcribe** — Whisper API returns transcript
3. **Extract** — Claude call #1: transcript → structured JSON
 - job type, scope items, dimensions, materials mentioned, access issues, timeline
4. **Enrich** — pull contractor profile (day rate, travel rate, markup %, VAT status) + pgvector search over past quotes for similar jobs
5. **Draft** — Claude call #2: structured job + contractor context → line-itemed quote draft with assumptions flagged ("assumed 2 days labour — confirm?")
6. **Review** — contractor edits in a simple mobile UI (this is the trust-building step; never auto-send)
7. **Render & send** — branded PDF + tracked link; log status (sent / viewed / accepted / declined)

Key data model (simplified)

contractors(id, company_name, company_number, trade, branding, vat_registered, vat_number,

day_rate, overtime_rate, callout_min, travel_rate, markup_pct)

team_members(id, contractor_id, name, role, day_rate)

merchants(id, name, base_url)

merchant_accounts(id, contractor_id, merchant_id, trade_discount_pct)

material_prices(id, merchant_id, sku, description, list_price, scraped_at)

customers(id, contractor_id, name, contact)

jobs(id, contractor_id, customer_id, source_audio_url, transcript, extracted_json, status)

quotes(id, job_id, line_items_json, total, pdf_url, status, sent_at, viewed_at)

invoices(id, quote_id, stripe_invoice_id, amount, due_date, status)

chase_events(id, invoice_id, channel, sent_at, template_used)

knowledge_chunks(id, contractor_id, embedding_vector, content, source_type)

Build Order (Claude Code task list)

Week 1 — Skeleton + quote pipeline

- ☐ Scaffold Next.js + Supabase project + auth
- ☐ Voice-first setup wizard: question flow, Whisper transcription, Claude extraction to structured fields, confirm-card UI
- ☐ Companies House lookup for company name/number verification
- ☐ Team members + rate variables in settings; logo upload
- ☐ Merchant selection + trade discount % capture (price scraping itself can slip to Week 3 — start with contractor-entered material prices)
- ☐ Audio capture component + upload to storage
- ☐ Whisper transcription endpoint
- ☐ Claude extraction prompt + JSON schema validation (zod)
- ☐ Quote drafting prompt using contractor profile
- ☐ Mobile quote editor (line items, totals, VAT)

- ☐ PDF generation with contractor branding
- ☐ Send quote via email with tracked link

Week 2 — Money + polish

- ☐ Quote accept/decline flow for the customer
- ☐ Stripe invoice creation from accepted quote (deposit + final)
- ☐ Webhook handling for payment events
- ☐ Chase sequence engine (day 3 / 7 / 14 templates, Haiku-generated, tone: firm but friendly)
- ☐ Basic dashboard: outstanding quotes, unpaid invoices

Week 3+ — Knowledge layer

- ☐ Embed past quotes/jobs into pgvector on save
 - ☐ Retrieval step in quote drafting ("similar past jobs")
 - ☐ Supplier/material price memory (contractor confirms prices; app remembers)
 - ☐ Learning loop: when a contractor edits a draft, store the delta to improve future drafts
-

Guardrails & Principles

- **Human in the loop, always.** Drafts are never sent without contractor review — trust is the product.
- **Flag assumptions explicitly** in every AI-generated quote (labour days, material quantities).
- **Money accuracy is non-negotiable:** all arithmetic done in code, not by the LLM. The LLM proposes line items; the app computes totals and VAT.
- **Keep it light.** No feature that requires training the contractor. If it needs a manual, cut it.
- **Data ownership:** each contractor's knowledge layer is theirs; easy export from day one.

Validation Plan (parallel to build)

- Recruit 3–5 real tradespeople (family contacts first) as design partners
- Watch them quote a real job with the tool; measure time saved and edit volume
- Pricing hypothesis to test: ~£30–50/month flat, or free quoting + small % on payments collected
- Kill criteria for Phase 1: if design partners won't use it for a second job unprompted, rework before building Phase 2